# Estructuras de Datos

2026
Segundo Cuatrimestre

# REPASO


AFA
FIFA WORLD CUP
Qatar2022
FRANCE
FFF
23 E. MARTÍNEZ
26 MOLINA
13 ROMERO
19 OTAMENDI
3 TAGLIAFICO
7 DE PAUL
24 FERNÁNDEZ
20 MAC ALLISTER
10 MESSI (C)
9 ÁLVAREZ
11 DI MARÍA
SUPLENTES
1. ARMANI, 12.RULLI, 2. FOYTH, 4. MONTIEL,
5. PAREDES, 6. PEZZELLA, 8. ACUÑA,
14. PALACIOS, 15. CORREA, 16. ALMADA,
17. GÓMEZ, 18. RODRÍGUEZ, 21. DYBALA,
22. L. MARTÍNEZ, 25. LI. MARTÍNEZ
DT: LIONEL SCALONI
10
86
78
PedidosYa
BINANCE
Schneider
Aerolíneas Argentinas
BETWARRIOR
KAVAK
Duelbits
KONAMI

# Diccionario esencial de la lengua española

Escriba aquí la palabra

áéíóúüñ

Debe tenerse en cuenta que esta versión electrónica da acceso al texto del *Diccionario esencial de la lengua española*, publicado en 2006.

Ver definición de *casa* en la edición actual del *Diccionario de la lengua española*

**casa.** F. **1.** Edificio para habitar. *Una casa de ocho plantas.* ‖ **2.** Edificio de una o pocas plantas destinado a vivienda unifamiliar, en oposición a *piso. Quieren vender el piso y comprarse una casa.* ‖ **3. piso** (‖ vivienda). *Mi casa está en el 3.º C.* ‖ **4.** Edificio, mobiliario, régimen de vida, etc., de alguien. *Echo de menos las comodidades de casa.* ‖ **5. familia** (‖ grupo de personas que viven juntas). *La administración de cada casa.* ‖ **6.** Descendencia o linaje que tiene un mismo apellido y viene del mismo origen. *Casa de los Ríos.* ‖ **7.** Establecimiento industrial o mercantil. *Esta casa es la más antigua en su ramo.* ‖ **8.** Institución de carácter sociocultural y recreativo que agrupa a personas con vínculos geográficos o intereses comunes. *Casa de Galicia. Casa de la cultura.* ‖ **9.** Sede de esta institución. ‖ **10.** hist. Conjunto de estados, vasallos y rentas que poseía un señor. ‖ **11.** pl. *Chile.* **casa** principal de un fundo. ‖ **~ abierta.** F. Domicilio, estudio o despacho de quien ejerce una profesión, un arte o una industria. ‖ **~ civil.** F. En España, conjunto de personas que tenían a su cargo los servicios no militares del palacio o residencia del jefe del Estado. ‖ **~ consistorial.** F. **casa** de la villa o ciudad adonde concurren los concejales de su ayuntamiento a celebrar sus juntas. ‖ **~ cuartel.** F. Puesto o destacamento permanente de un cuerpo armado dotado de viviendas destinadas a los individuos de este cuerpo y sus familias. ‖ **~ cuna.** F. **inclusa.** ‖ **~ cural.** F. **casa parroquial.** ‖ **~ de altos.** F. *Am.* Vivienda que tiene dos o más pisos sobre la planta baja. ‖ **~ de baños.** F. Establecimiento en que se tienen baños para el servicio público. ‖ **~ de beneficencia.** F. Hospital, hospicio o asilo. ‖ **~ de campo.** F. La que está fuera de poblado y sirve para cuidar del cultivo, para recrearse o para ambos objetos a la vez. ‖ **~ de citas.** F. Aquella en que se facilita, clandestinamente, y a cambio de dinero, habitación para las relaciones sexuales. ‖ **~ de comidas.** F. **figón.** ‖ **~ de Dios.** F. Templo o iglesia. ‖ **~ de empeño,** o **~ de empeños.** F. Establecimiento donde se presta dinero mediante la entrega condicionada de alhajas o ropas u otros bienes muebles, en prenda. ‖ **~ de huéspedes.** F. Aquella en que, mediante cierto precio, se da estancia y comida, o solo alojamiento, a algunas personas. ‖ **~ de juego.** F. La que está destinada a la explotación de juegos de azar. ‖ **~ de labor,** o **~ de labranza.** F. Aquella en que habitan los labradores y en la que tienen sus ganados y aperos. ‖ **~ de lenocinio.** F. La de prostitutas. ‖ **~ de locos.** F. **1. manicomio.** ‖ **2.** coloq. Lugar donde hay mucho bullicio, inquietud y falta de autoridad. ‖ **~ del rey.** F. **1. casa real.** ‖ **2.** En España, conjunto de personas civiles y militares que asisten al Rey en el desempeño de sus funciones. ‖ **~ del Señor.** F. **casa de Dios.** ‖ **~ de malicia.** F. hist. Cada una de las que, con cabida para alojar dos familias, se edificaban en Madrid durante el siglo XVII, sobre traza arquitectónica en fachada de un solo piso, rehuyendo así la carga de aposento. ‖ **~ de mancebía.** F. **casa de lenocinio.** ‖ **~ de moneda.** F. La destinada a fundir, fabricar y acuñar moneda. ‖ **~ de oración.** F. **casa de Dios.** ‖ **~ de postas.** F. hist. Parada donde tomaban caballos de refresco los correos y quienes viajaban en posta. ‖ **~ de préstamos.** F. **casa de empeño.** ‖ **~ de prostitución.** F. **casa de lenocinio.** ‖ **~ de pupilos.** F. **casa de huéspedes.** ‖ **~ de putas.** F. **1.** coloq. **casa de lenocinio.** ‖ **2.** coloq. Lugar de gran desorden. ‖ **~ de recreo.** F. La situada en el campo como lugar de descanso y distracción. ‖ **~ de socorro.** F. Establecimiento benéfico donde se prestan los primeros auxilios facultativos a heridos o víctimas de una enfermedad repentina. ‖ **~ de tócame Roque.** F. coloq. Aquella en que vive mucha gente y hay mala dirección y el consiguiente desorden. ‖ **~ de tolerancia.** F. **casa de lenocinio.** ‖ **~ de trato.** F. **casa de lenocinio.** ‖ **~ de vecindad.** F. La que contiene muchas viviendas reducidas, por lo común con acceso a patios y corredores. ‖ **~ dezmera,** o **~ excusada.** F. hist. La principal de cada parroquia que, por privilegio pontificio, pagaba sus diezmos a la real Hacienda en vez de pagarlos a la Iglesia. ‖ **~ fuerte.** F. **1.** La fabricada para habitar en ella, con medios para defenderse de los enemigos. ‖ **2.** La muy acaudalada. ‖ **~ grande.** F. **casa** aristocrática o importante por su riqueza. ‖ **~ militar.** F. En España, conjunto de militares que se hallaban como ayudantes al servicio inmediato del jefe del Estado. ‖ **~ mortuoria.** F. **casa** donde recientemente ha muerto alguna persona. ‖ **~ parroquial.** F. La destinada a vivienda del párroco y que, generalmente, es propiedad de la Iglesia. ‖ **~ paterna.** F. **casa** de los padres. ‖ **~ pública.** F. **casa de lenocinio.** ‖ **~ real.** F. **1. palacio** (‖ **casa** de los reyes). ‖ **2.** Personas reales y conjunto de sus familias. ‖ **~ rectoral.** F. **casa parroquial.** ‖ **~ rodante.** F. *Am.*

# Implementaciones TDAs Mapeo y Diccionario

Hasta ahora programamos nuestros TDAs mapeo y diccionario utilizando listas (doblemente enlazadas, con enlace al final y centinelas)

| Lista con Nodo<E> Doblemente Enlazado + Posición | | |
|---|---|---|
| Métodos del TDA | ¿Se resuelve inmediato? | ¿Salimos a buscar algo? |
| first() | Si | No |
| last() | Si | No |
| prev(p) | Si | No |
| next(p) | Si | No |
| set( p, e ) | Si | No |
| addFirst(e) | Si | No |
| addLast(e) | Si | No |
| addBefore(p, e) | Si | No |
| addAfter(p, e) | Si | No |
| remove(p) | Si | No |

# Implementaciones TDAs Mapeo y Diccionario

Hasta ahora programamos nuestros TDAs mapeo y diccionario utilizando listas (doblemente enlazadas, con enlace al final y centinelas)

# Implementaciones TDAs Mapeo y Diccionario

Hasta ahora programamos nuestros TDAs mapeo y diccionario utilizando listas (doblemente enlazadas, con enlace al final y centinelas)

Bajo esta implementación de lista ¿se resuelve inmediatamente una operación que tiene que ver si un elemento está en la lista?

| Lista con Nodo<E> Doblemente Enlazado + Posición | | |
|---|---|---|
| Métodos | ¿Se resuelve inmediato? | ¿Salimos a buscar algo? |
| get(k) / find(e) | | |

# Implementaciones TDAs Mapeo y Diccionario

Hasta ahora programamos nuestros TDAs mapeo y diccionario utilizando listas (doblemente enlazadas, con enlace al final y centinelas)

Bajo esta implementación de lista ¿se resuelve inmediatamente una operación que tiene que ver si un elemento está en la lista?

# Implementaciones TDAs Mapeo y Diccionario

Hasta ahora programamos nuestros TDAs mapeo y diccionario utilizando listas (doblemente enlazadas, con enlace al final y centinelas)

Bajo esta implementación de lista ¿se resuelve inmediatamente una operación que tiene que ver si un elemento está en la lista?

Tiene sentido pensar que nunca se va a poder resolver en forma inmediata esta búsqueda porque es necesario revisar entre todos los elementos si está el elemento buscado.

# Implementaciones TDAs Mapeo y Diccionario

Hasta ahora programamos nuestros TDAs mapeo y diccionario utilizando listas (doblemente enlazadas, con enlace al final y centinelas)

Bajo esta implementación de lista ¿se resuelve inmediatamente una operación que tiene que ver si un elemento está en la lista?

Tiene sentido pensar que nunca se va a poder resolver en forma inmediata esta búsqueda porque es necesario revisar entre todos los elementos si está el elemento buscado. **¿Es realmente necesario?**

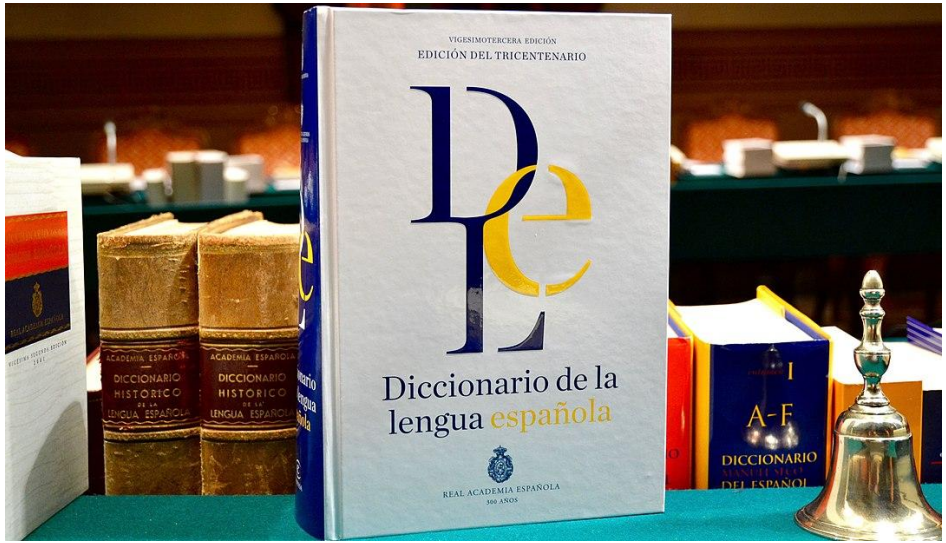
# TABLAS DE HASH

# Tabla de Hash

- La tabla de Hash (o el *hashing*) es una estrategia de implementación, NO es un TDA
  - En particular, es una forma de implementar estructuras que vamos a usar con los TDAs Mapeo y Diccionario

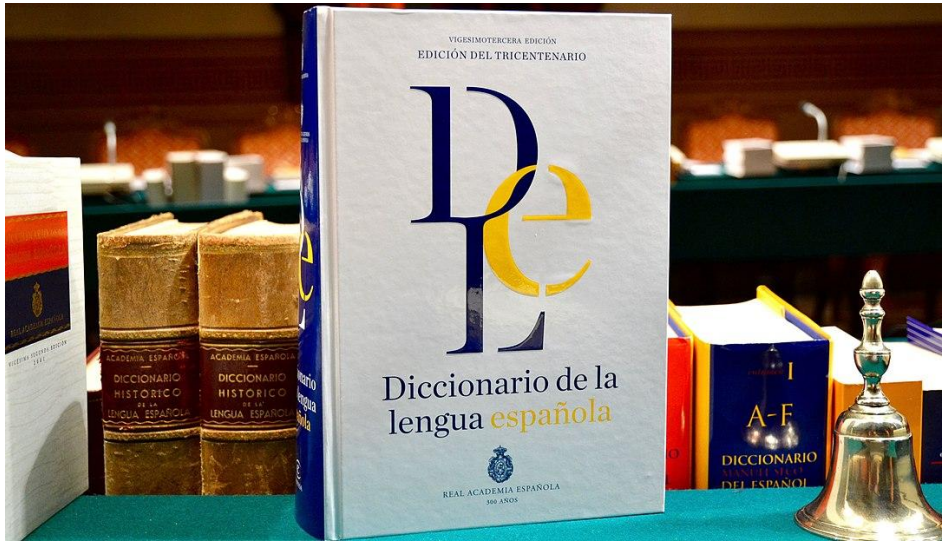
# Tabla de Hash

Desde un punto de vista conceptual, todo lo que mantiene unido a ciertos datos conforma una estructura de datos



# Tabla de Hash

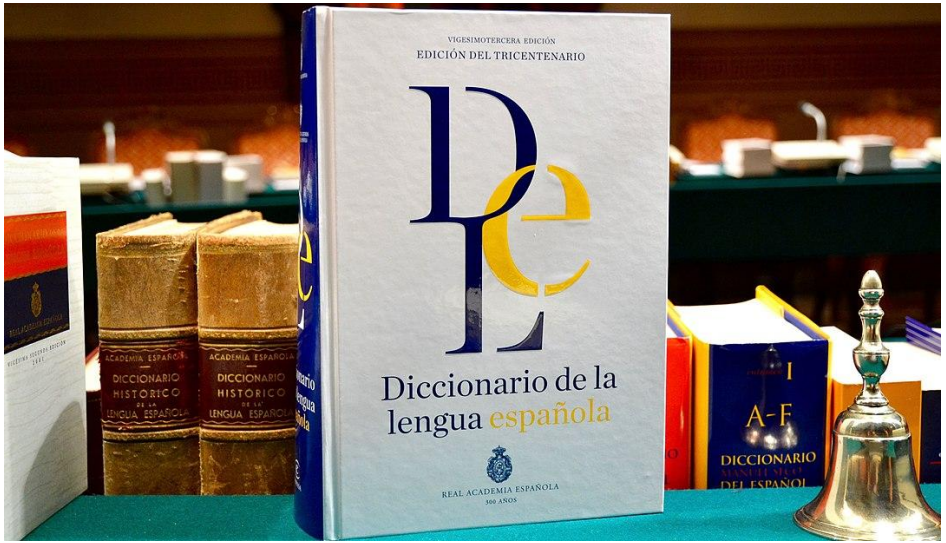
Buscar una palabra en un diccionario (real) ¿se resuelve en forma inmediata? ¿en necesario buscar entre todas las palabras?



# Tabla de Hash

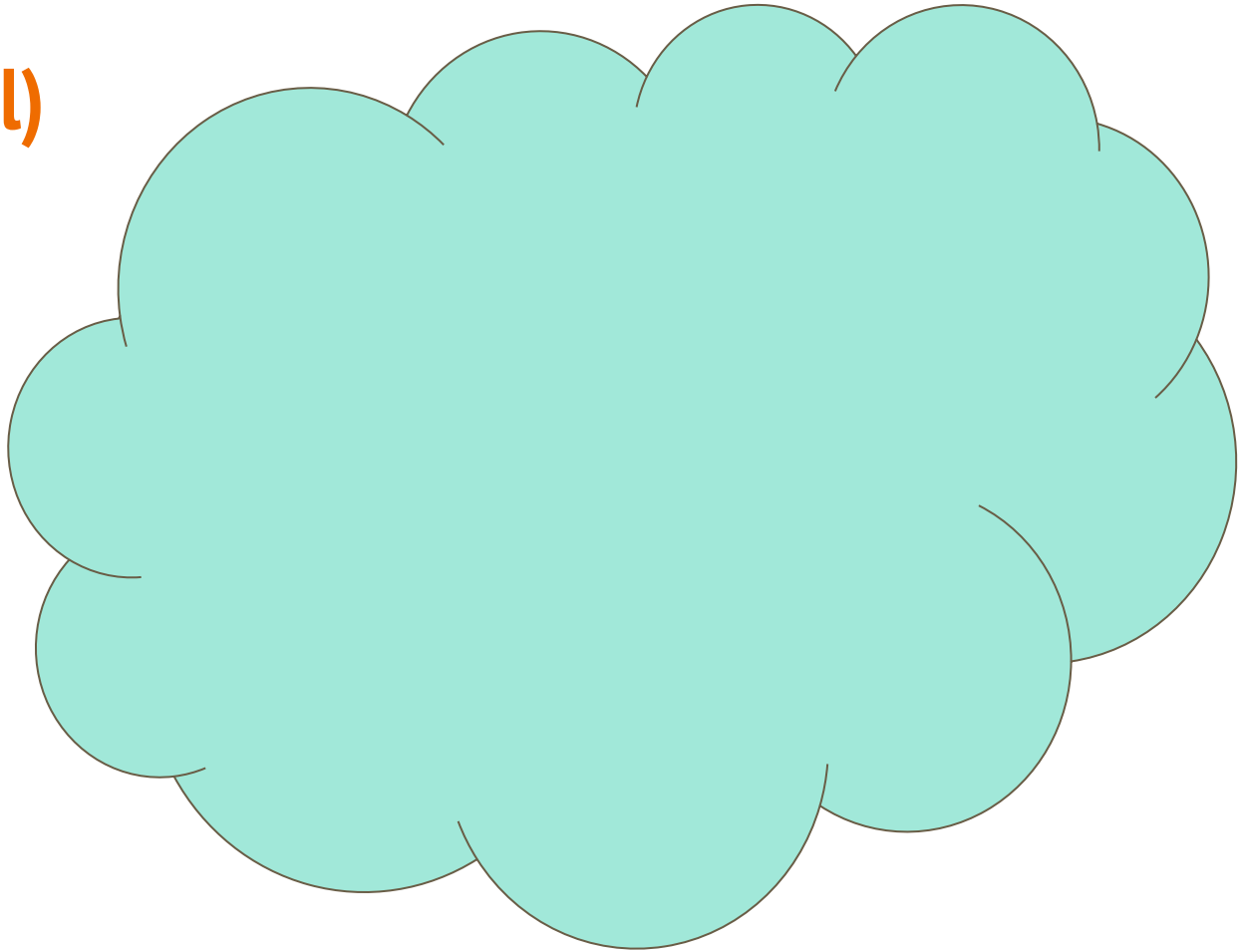
Buscar una palabra en un diccionario (real) ¿se resuelve en forma inmediata? ¿en necesario buscar entre todas las palabras?



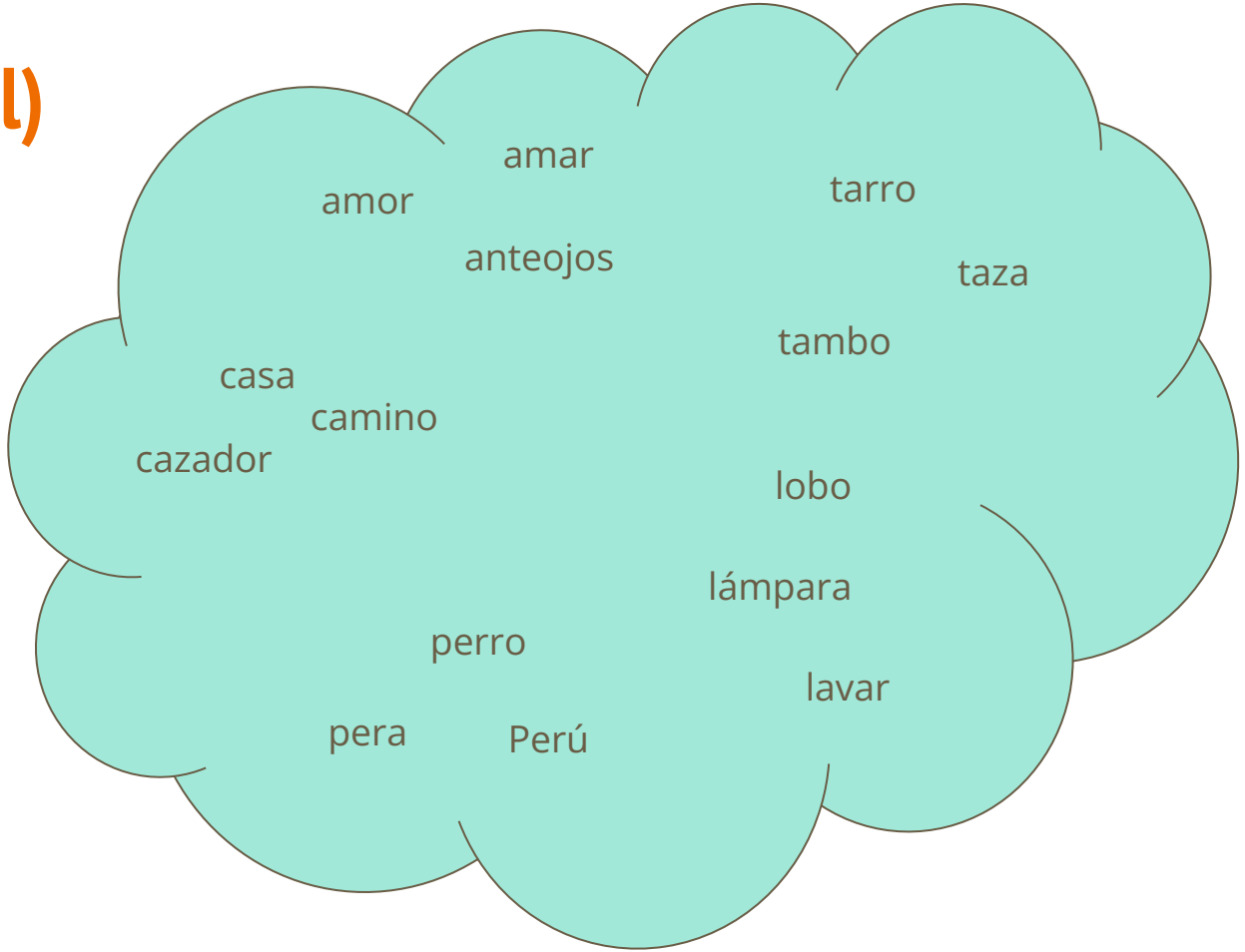


# Diccionario (real)

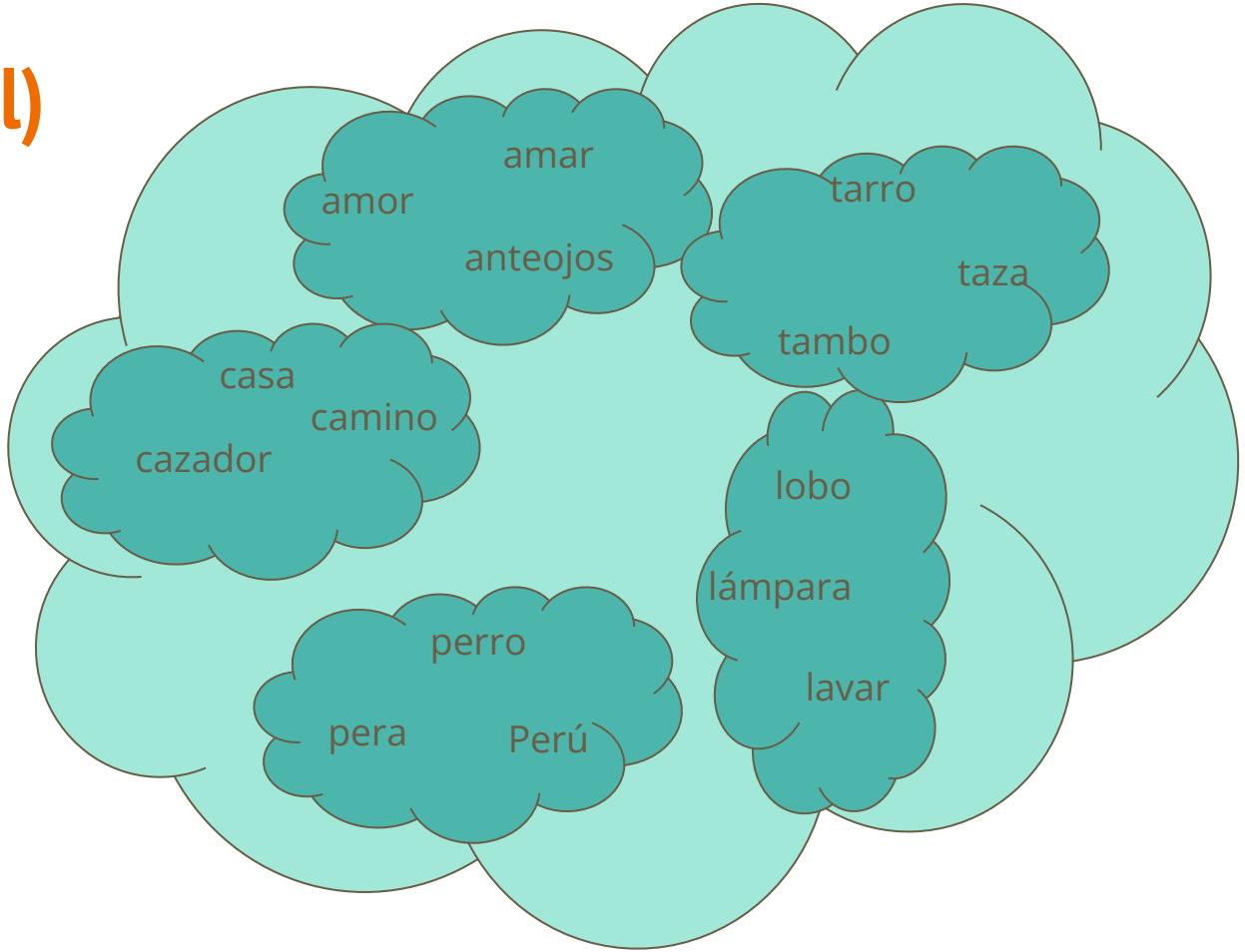
# Diccionario (real)



# Diccionario (real)

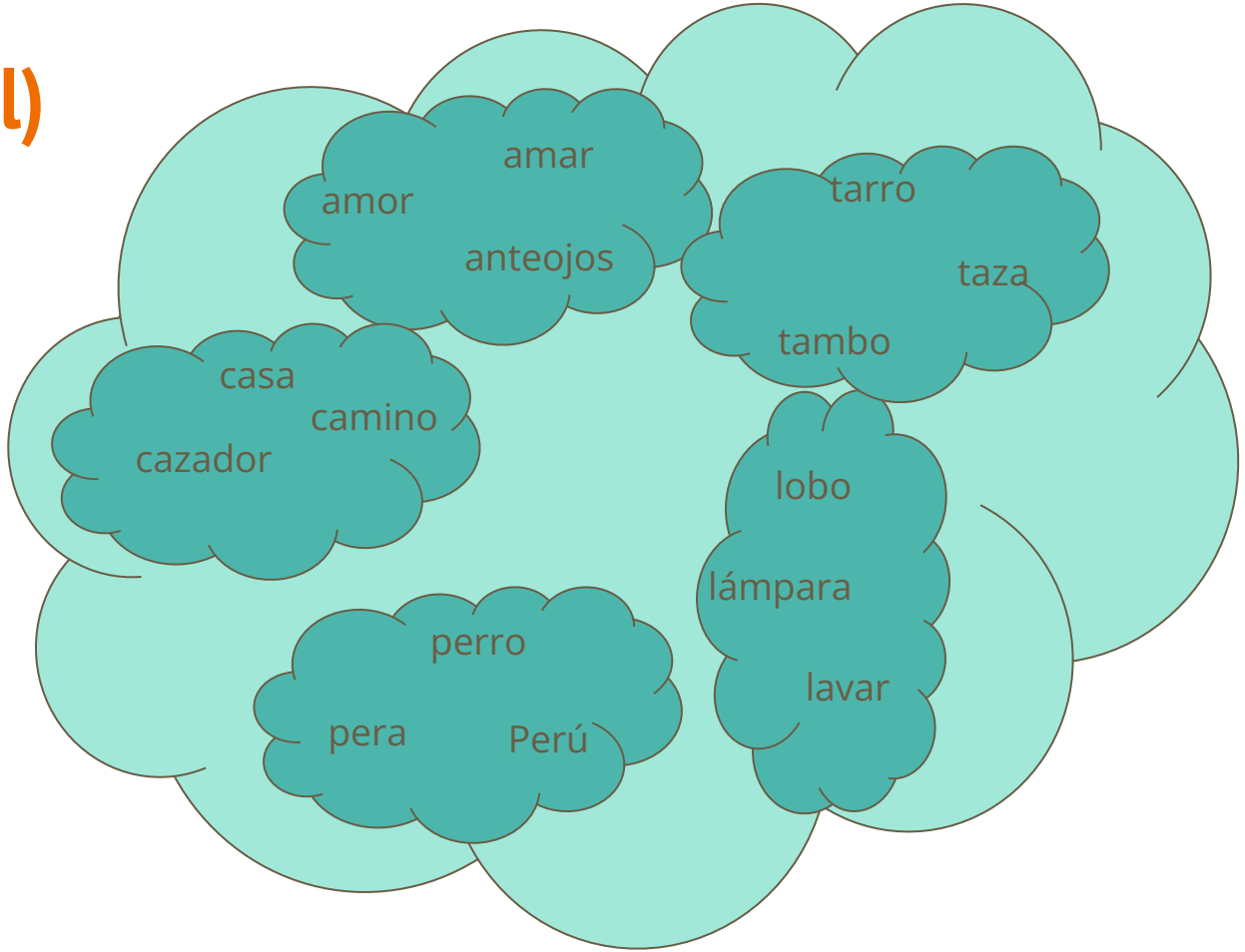


# Diccionario (real)



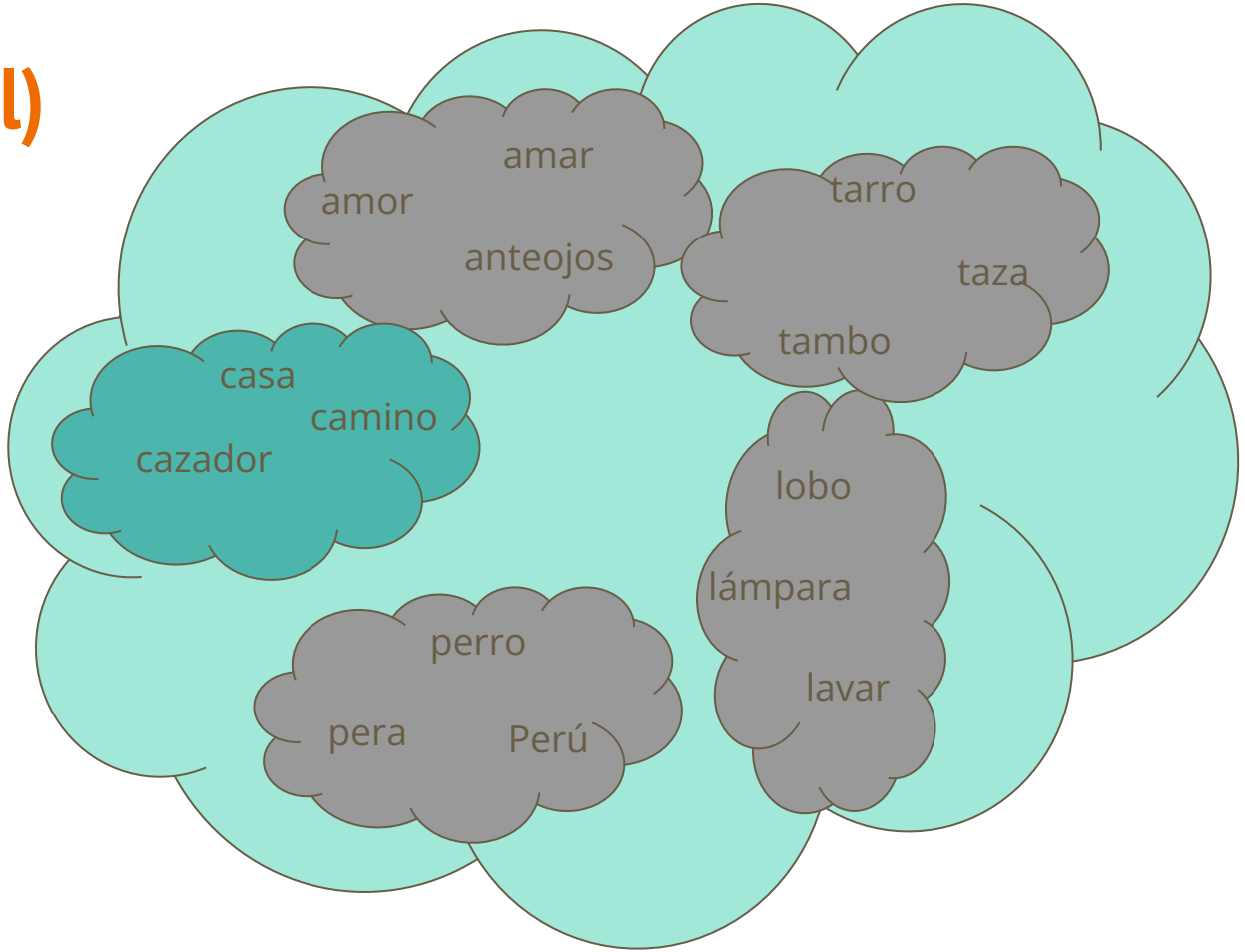
# Diccionario (real)

buscar(camino)



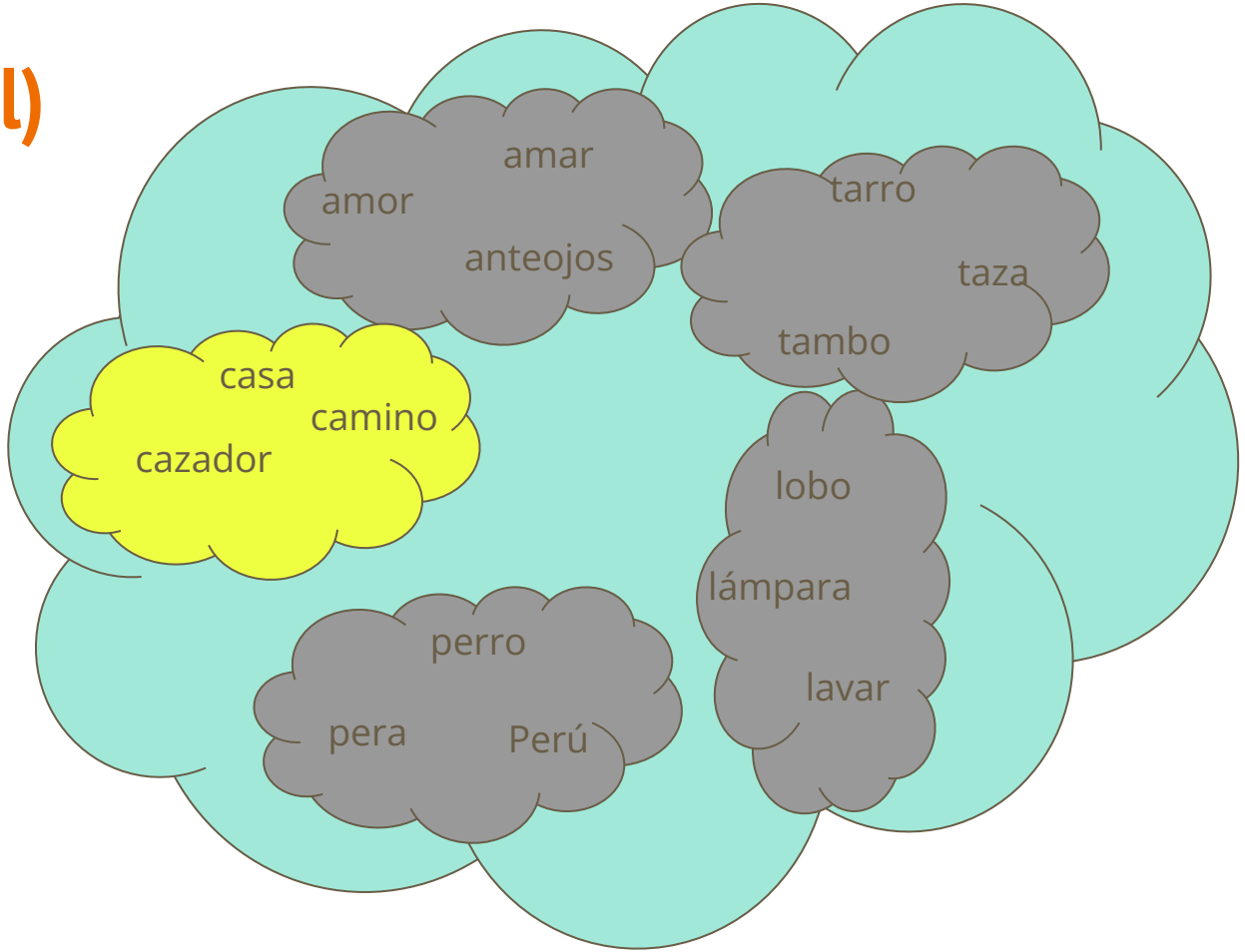
# Diccionario (real)

buscar(camino)



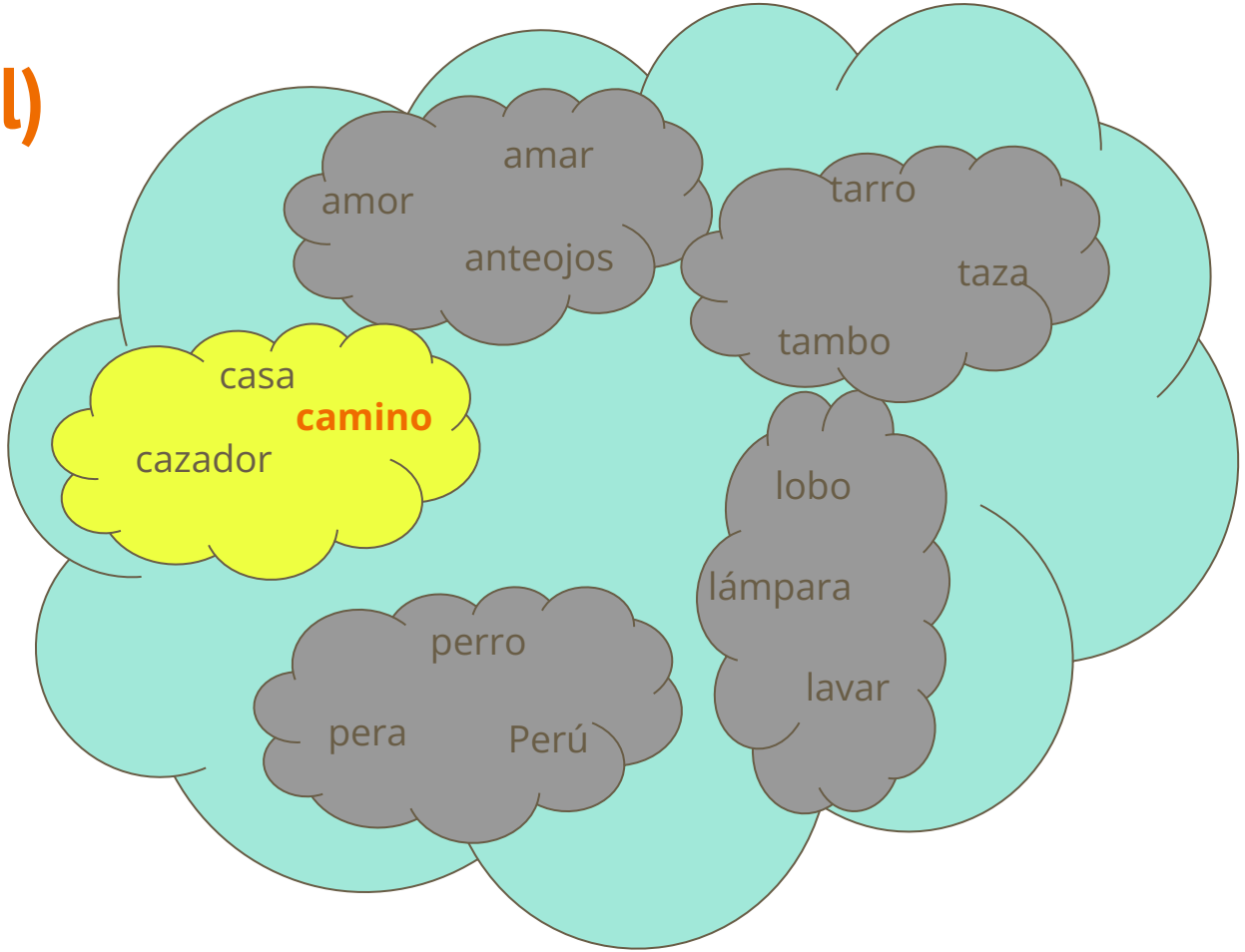
# Diccionario (real)

buscar(camino)



# Diccionario (real)

buscar(camino)



# Tablas de Hash

La idea es buscar una representación (implementación) de una colección de elementos, de tal manera que las operaciones de búsqueda, inserción y eliminación no requieran salir a buscar cosas. Especialmente la búsqueda.

# Tablas de Hash

La idea es buscar una representación (implementación) de una colección de elementos, de tal manera que las operaciones de búsqueda, inserción y eliminación no requieran salir a buscar cosas. Especialmente la búsqueda.

Y si tenemos que salir a buscar algo, que sea sobre un subconjunto significativamente menor que el total de los elementos

# Tablas de Hash

Tabla de Hash (o Tabla Hash) es una estrategia para lograr esto.

# Tablas de Hash

Tabla de Hash (o Tabla Hash) es una estrategia para lograr esto.

Tenemos dos variantes de la estrategia, Hash Abierto y Hash Cerrado.

Hash Abierto: *separate chaining*

Hash Cerrado: *open addressing*

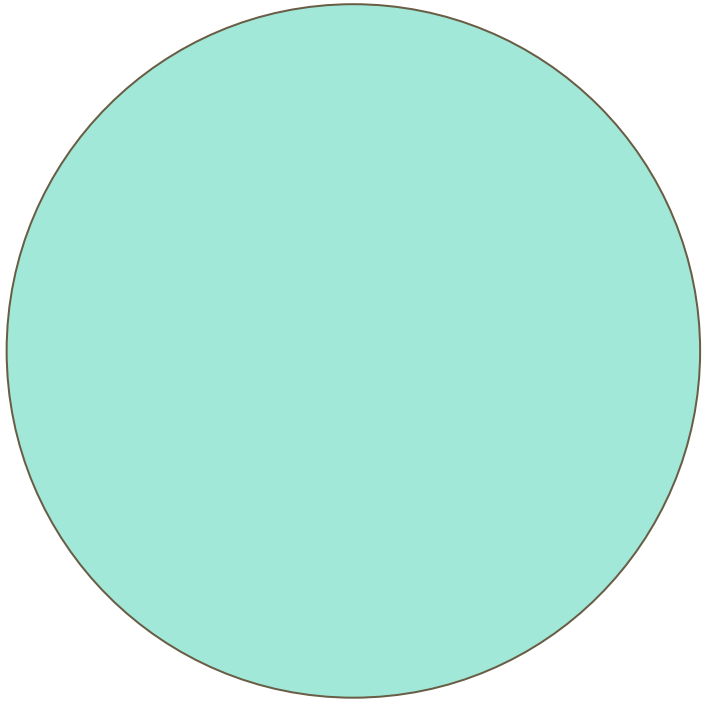
# Tablas de Hash

Tabla de Hash (o Tabla Hash) es una estrategia para lograr esto.

Tenemos dos variantes de la estrategia, **Hash Abierto** y Hash Cerrado.

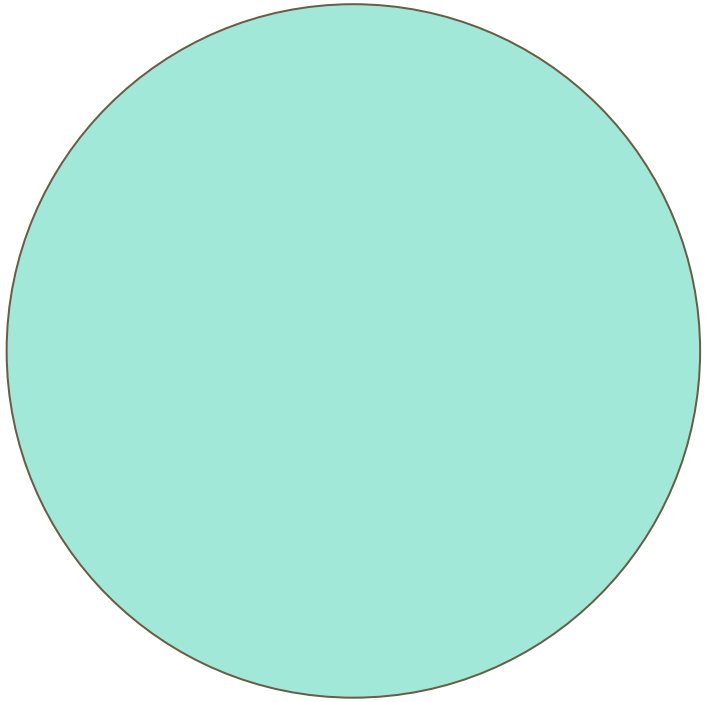
# Tabla de Hash Abierto

En lugar de tener una colección de elementos

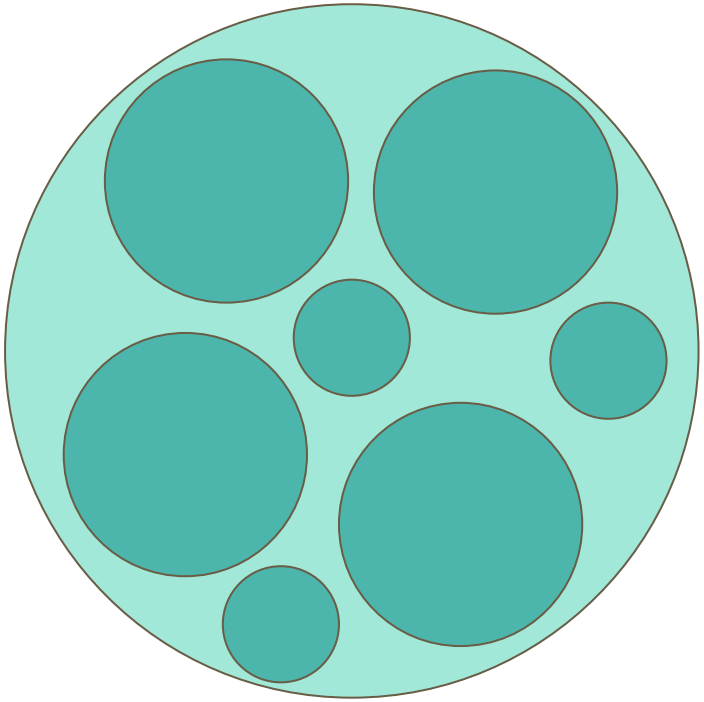


# Tabla de Hash Abierto

En lugar de tener una colección de elementos



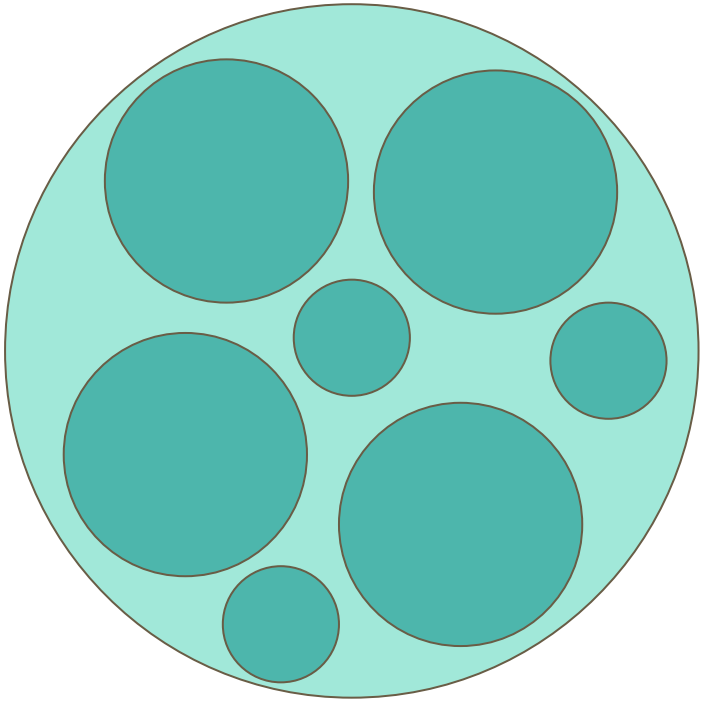
Particionamos la colección en *buckets* o *cubetas*



# Tabla de Hash Abierto

Esto lo hacemos de manera tal que, dado un elemento podemos saber en qué cubeta debería estar.

Particionamos la colección en *buckets* o *cubetas*

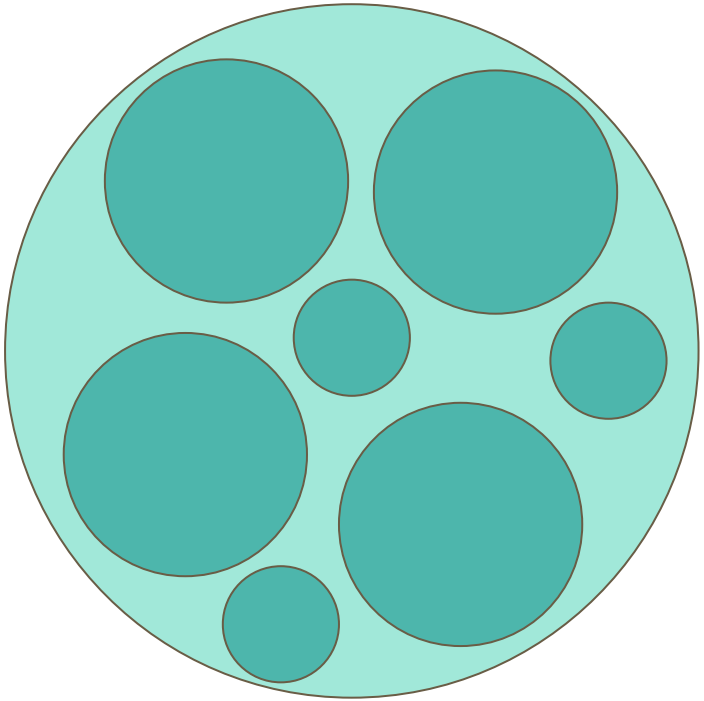


# Tabla de Hash Abierto

Esto lo hacemos de manera tal que, dado un elemento podemos saber en qué cubeta debería estar.

Si podemos lograr esto, todas las interacciones se limitan sólo a esa cubeta y no a toda la colección de elementos (buscar, insertar, eliminar)

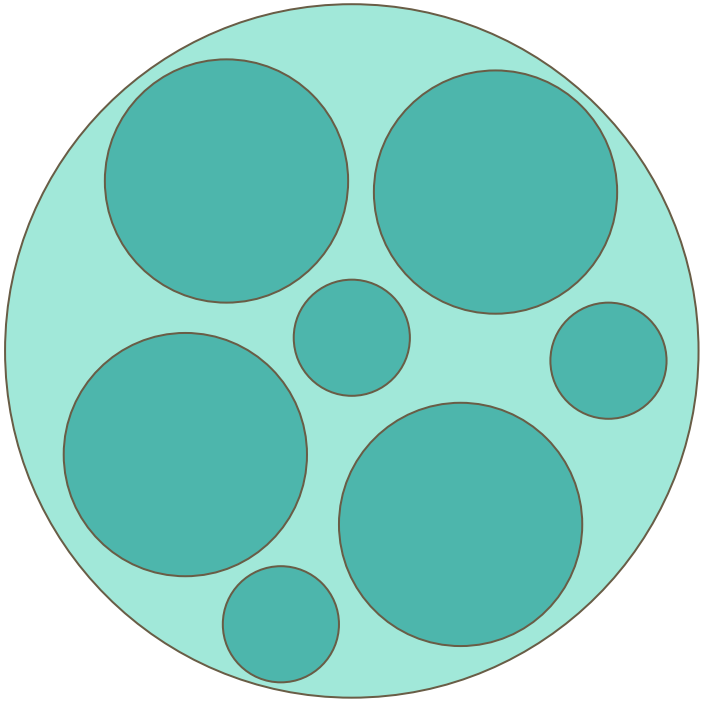
Particionamos la colección en *buckets* o *cubetas*



# Tabla de Hash Abierto

Si logramos una buena distribución de elementos entre las cubetas, cada cubeta debería tener *n*/*#cubetas* de elementos. Donde *n* es la cantidad de elementos de la colección.

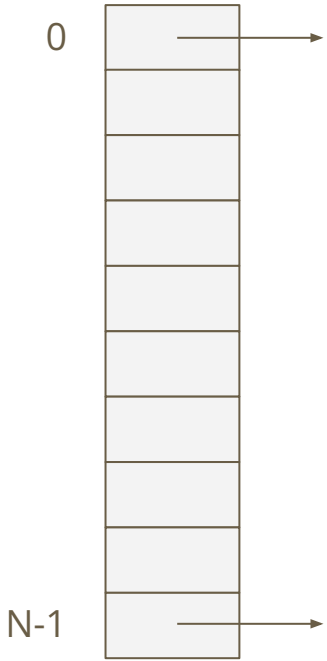
Particionamos la colección en *buckets* o *cubetas*



# Tabla de Hash Abierto

¿Cómo implementamos este concepto?

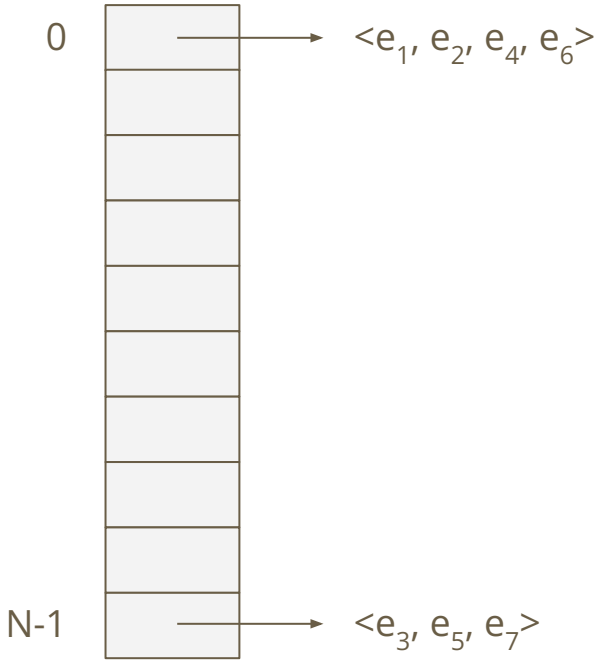
# Tabla de Hash Abierto




Arreglo de Cubetas

El arreglo mantiene N cubetas.

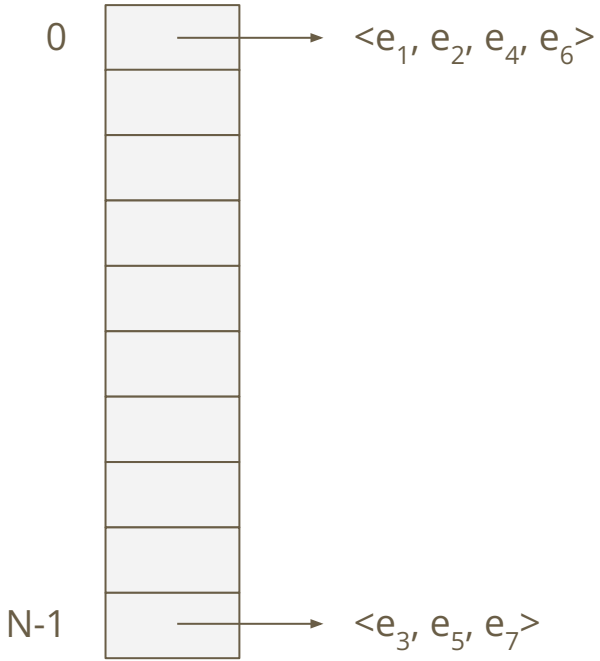
# Tabla de Hash Abierto




Arreglo de Cubetas

El arreglo mantiene N cubetas.
Cada cubeta es una lista de elementos

# Tabla de Hash Abierto

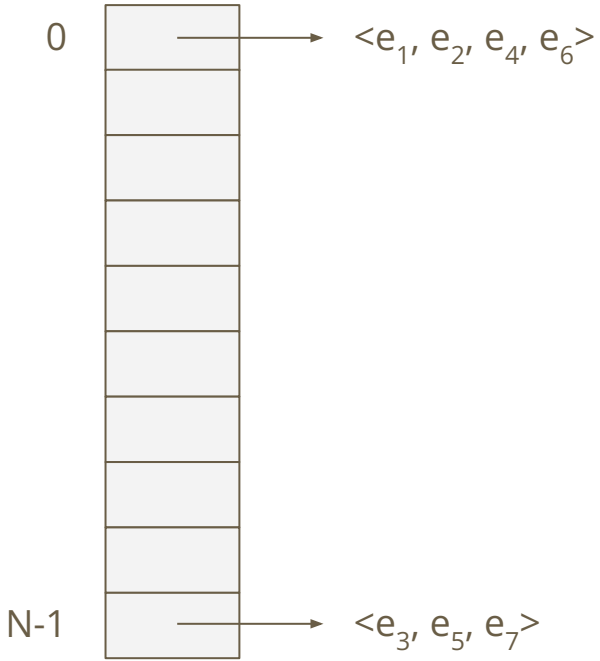



Necesitamos una forma de asociar cubetas con elementos. Es decir, dado el elemento $e_x$, ¿en qué cubeta debería estar/ir?

Supongamos edades de personas de 0 a 99
Agrupo las edades según su decena

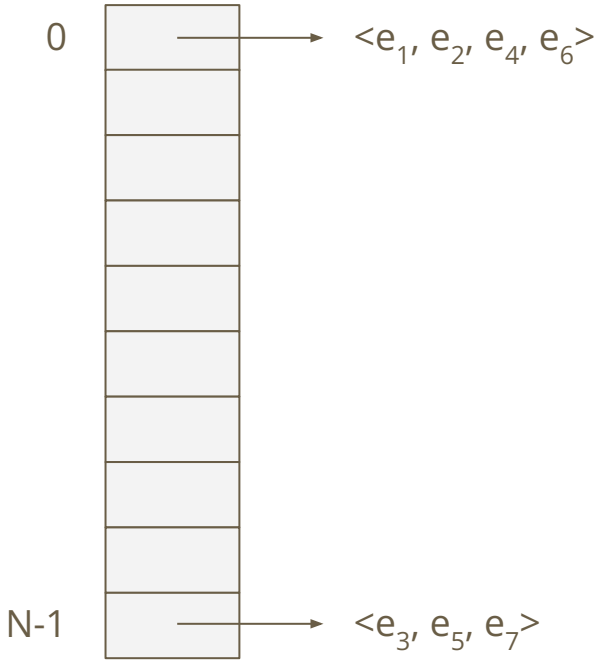
¿Qué problema tiene esto?

# Tabla de Hash Abierto




Vamos a introducir la función *hash(algo).* Tal que, hash(algo) me retorna la cubeta donde ese algo debería estar.

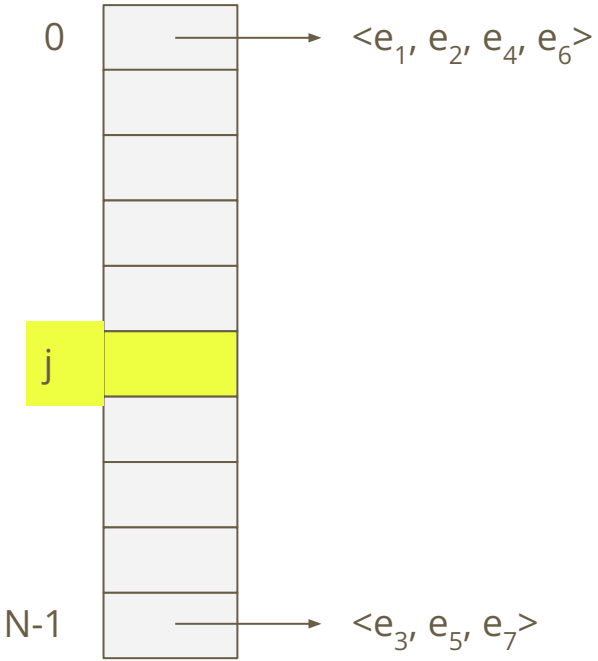
# Tabla de Hash Abierto




Necesitamos una forma de asociar cubetas con elementos. Es decir, dado el elemento $e_x$, ¿en qué cubeta debería estar/ir?

$hash(e_x) = j$

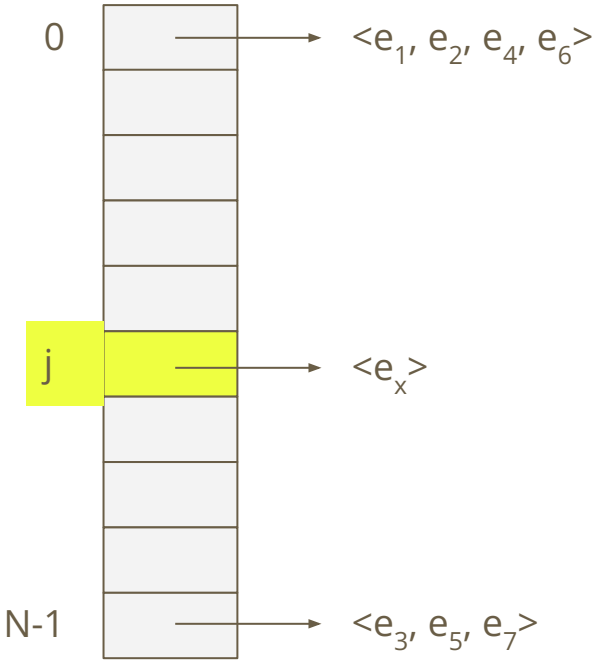
# Tabla de Hash Abierto




Necesitamos una forma de asociar cubetas con elementos. Es decir, dado el elemento $e_x$, ¿en qué cubeta debería estar/ir?

$hash(e_x) = j$

# Tabla de Hash Abierto




Necesitamos una forma de asociar cubetas con elementos. Es decir, dado el elemento $e_x$, ¿en qué cubeta debería estar/ir?

$hash(e_x) = j$

# Tabla de Hash Abierto para Mapeo/Diccionario

# Tabla de Hash Abierto para Mapeo/Diccionario

El elemento que guardamos son las entradas (k,v) y la función *hash* se hace sobre las claves.

# Tabla de Hash Abierto para Mapeo/Diccionario

El elemento que guardamos son las entradas (k,v) y la función *hash* se hace sobre las claves.

Dada una clave k y un valor v, hash(k) es un entero en el intervalo [0,N-1] tal que la entrada (k,v) se inserta en la cubeta A[hash(k)]. Para simplificar vamos a escribir h(k).

# Tabla de Hash Abierto para Mapeo/Diccionario

Si dos entradas $e_1$ y $e_2$, con claves $k_1$ y $k_2$, caen en la misma cubeta, decimos que hay una colisión porque $h(k_1) = h(k_2)$.

Esto no es algo necesariamente negativo, sólo describe una situación.

# Tabla de Hash Abierto para Mapeo/Diccionario

Una función de hash h es “**buena**” si h distribuye las claves uniformemente en el intervalo [0,C-1]. Intuitivamente, todas las cubetas tienen aproximadamente el mismo tamaño.

# Tabla de Hash Abierto para Mapeo/Diccionario

Una función de hash h es “**buena**” si h distribuye las claves uniformemente en el intervalo [0,C-1]. Intuitivamente, todas las cubetas tienen aproximadamente el mismo tamaño.

Si hay almacenadas n entradas distribuidas en C cubetas (de forma uniforme), entonces el tamaño de cada cubeta es n/C.

Para favorecer la distribución de claves uniformemente, buscamos que C sea primo (ya vamos a ver el porqué)

# Tabla de Hash Abierto para Mapeo/Diccionario

Si nuestra clave es numérica, (clave MOD C) es una posible función hash.

# Tabla de Hash Abierto para Mapeo/Diccionario

Si nuestra clave es numérica, (clave MOD C) es una posible función hash.

h(k) es k MOD C = 0 ... C - 1

# Tabla de Hash Abierto para Mapeo/Diccionario

Tracemos el siguiente ejemplo:

M  = new MapeoConHashAbierto<Integer,Character>(); //Supongamos que se implementa con 10 Cubetas

M.put( 5, ‘A’ )

M.put( 6, ‘B’ )

M.put( 3, ‘C’ )

M.put( 15, ‘D’ )

M.put( 25, ‘E’ )

M.put( 35, ‘F’ )

M.put( 56, ‘G’ )

M.put( 23, ‘H’ )

M.get(15) ; M.get(40); M.remove(15)

# Tabla de Hash Abierto

¿Cómo implementar la función *hash* cuando mi elemento no es un entero?

# Tabla de Hash Abierto

¿Cómo implementar la función *hash* cuando mi elemento no es un entero?

Lo primero que podemos hacer es, dado el elemento obtener un número entero asociado. A este número lo llamamos *código hash, o hash code.*

# Tabla de Hash Abierto

¿Cómo implementar la función *hash* cuando mi elemento no es un entero?

Lo primero que podemos hacer es, dado el elemento obtener un número entero asociado. A este número lo llamamos *código hash, o hash code.*

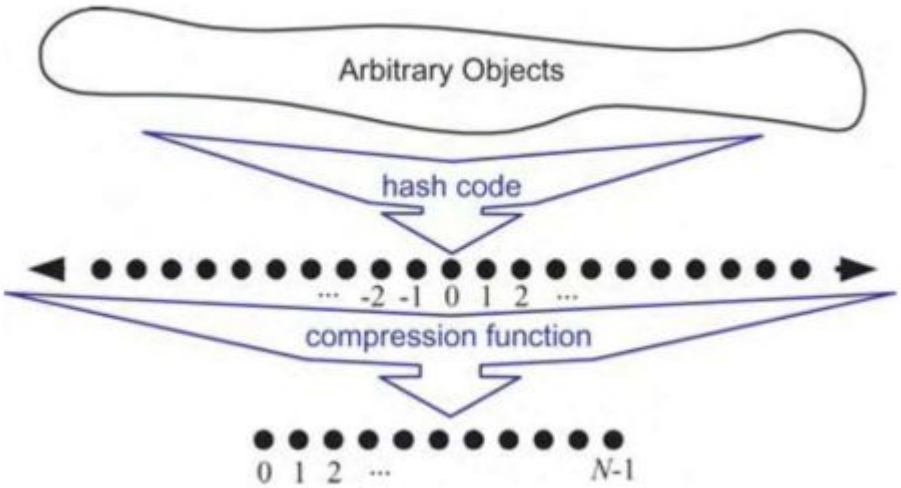
Luego aplicamos la función hash (MOD C) sobre el código hash.

# Tabla de Hash Abierto

¿Cómo implementar la función *hash* cuando mi elemento no es un entero?

Lo primero que podemos hacer es, dado el elemento obtener un número entero asociado. A este número lo llamamos *código hash, o hash code.*

Luego aplicamos la función hash (MOD C) sobre el código hash.



## Class Object

java.lang.Object

---

`public class Object`

Class `Object` is the root of the class hierarchy. Every class has `Object` as a superclass. All objects, including arrays, implement the methods of this class.

**Since:**

`JDK1.0`

**See Also:**

`Class`

### *Constructor Summary*

**Constructors**

| Constructor and Description |
| --- |
| `Object()` |

### *Method Summary*

**All Methods** | **Instance Methods** | **Concrete Methods**

| Modifier and Type | Method and Description |
| --- | --- |
| `protected Object` | `clone()`<br>Creates and returns a copy of this object. |
| `boolean` | `equals(Object obj)`<br>Indicates whether some other object is "equal to" this one. |
| `protected void` | `finalize()`<br>Called by the garbage collector on an object when garbage collection determines that there are no more references to the object. |
| `Class<?>` | `getClass()`<br>Returns the runtime class of this `Object`. |
| `int` | `hashCode()`<br>Returns a hash code value for the object. |
| `void` | `notify()`<br>Wakes up a single thread that is waiting on this object's monitor. |

# Tabla de Hash Abierto - Hash code

| All Methods | Instance Methods | Concrete Methods |
|---|---|---|

| Modifier and Type | Method and Description |
|---|---|
| protected `Object` | **`clone()`**<br>Creates and returns a copy of this object. |
| boolean | **`equals(Object obj)`**<br>Indicates whether some other object is "equal to" this one. |
| protected void | **`finalize()`**<br>Called by the garbage collector on an object when garbage collection determines |
| `Class<?>` | **`getClass()`**<br>Returns the runtime class of this `Object`. |
| int | **`hashCode()`**<br>Returns a hash code value for the object. |
| void | **`notify()`**<br>Wakes up a single thread that is waiting on this object's monitor. |

# Tabla de Hash Abierto - Hash code

| | |
|---|---|
| `int` | **`hashCode()`**<br>Returns a hash code value for the object. |

# Tabla de Hash Abierto - Hash code

La forma más sencilla es usar el método *public int hashCode();* heredado de Object.

**Cuidado**: En la implementación más simple *hashCode()* retorna la dirección de memoria del objeto con lo que dos objetos iguales (en el sentido de equals) tendrían distinto hashCode.

La clase String redefine el método *hashCode()*

# Tabla de Hash Abierto - Hash code

```
public class Persona {
    private String nombre;
    private int edad;
    private String mail;

            public Persona(String nombre, int edad, String mail) {
                this.nombre = nombre;
                this.edad = edad;
                this.mail = mail;    }

        public int hashCode() {
                return (dni + nombre + email).hashCode();       }
```

# Tabla de Hash Abierto - Hash code

```
public class Persona {
    private String nombre;
    private int edad;
    private String mail;

            public Persona(String nombre, int edad, String mail) {
                this.nombre = nombre;
                this.edad = edad;
                this.mail = mail;    }

        public int hashCode() {
                return (dni + nombre + email).hashCode();     }
```

// Este hashCode es el de String

# Tabla de Hash Abierto - Función de compresión

- Dado i = k.hashCode()
- Implementaciones para calcular el número de cubeta a partir de i
  - Método de la división: i mod N (N idealmente primo)
    - El motivo de porqué usar primos es en los casos que haya algún tipo de patrón en la entrada (hash codes)
      - Por ejemplo, posiciones en memoria de enteros grandes (long/bigint) ocupan 64 bits (8 bytes)
      - ¿Qué pasa si uso N =4?
      - Los números que menos factores comunes tienen son los primos

# Tabla de Hash Abierto - Función de compresión

- Dado i = k.hashCode()
- Implementaciones para calcular el número de cubeta a partir de i
  - Método de la división: i mod N (N idealmente primo)
    - El motivo de porqué usar primos es en los casos que haya algún tipo de patrón en la entrada (hash codes)
      - Por ejemplo, posiciones en memoria de enteros grandes (long/bigint) ocupan 64 bits (8 bytes)
      - ¿Qué pasa si uso N =4?
      - Los números que menos factores comunes tienen son los primos
  - Metodo MAD (multiply, add and divide)
    - [(a*i + b) mod p] mod N
      - a y b son enteros entre [0, p - 1], p es primo y p > N

# Tabla de Hash Abierto para Mapeo/Diccionario



**Clase Mapeo<K,V>**

Atributos:

```
Map<K,V> [] A

int n // n es size, la cantidad de entradas del mapeo

int N ← 13  // o cualquier otro número primo
```

Constructor Mapeo()

```
n ← 0

A ← (Map<K,V> []) new MapeoConLista[N]

for i ← 0 to N-1 do

        A[i] ← new MapeoConLista<K,V>()
```

**Algoritmo** get(k)
{ retorna el valor asociado con la clave k en el mapeo, o null si no hay una entrada con clave k en el mapeo }
    **return** A[h(k)].get(k) { Delega en el get del mapeo implementado con lista. }

**Algoritmo** put(k, v)
{ Si hay una entrada en el mapeo con clave k, reemplaza el valor con v y retorna el viejo valor. Sino agrega la entrada (k,v) y retorna null }
    t ← A[h(k)].put(k,v) { Delega en el put del mapeo implementado con lista. }
    if t = null then { k es una nueva clave }
        n ← n+1 { incremento la cantidad de entradas en el mapeo }
    **return** t

**Algoritmo** remove(k)
{ Retorna el valor asociado con la clave k del mapeo, o null si no hay una entrada con clave k }
    t ← A[h(k)].remove(k) { Delega en el mapeo implementado con lista. }
    if t ≠ null then { k fue encontrado }
        n  n – 1 { decremento la cantidad de entradas del mapeo }
    **return** t

**Algoritmo** get(k)
{ retorna el valor asociado con la clave k en el mapeo, o null si no hay una entrada con clave k en el mapeo }

```
return A[h(k)].get(k) { Delega en el get del mapeo implementado con lista. }
```

**Algoritmo** put(k, v)
{ Si hay una entrada en el mapeo con clave k, reemplaza el valor con v y retorna el viejo valor. Sino agrega la entrada (k,v) y retorna null }

```
t ← A[h(k)].put(k,v) { Delega en el put del mapeo implementado con lista. }
if t = null then { k es una nueva clave }
    n ← n+1 { incremento la cantidad de entradas en el mapeo }
return t
```

**Algoritmo** remove(k)
{ Retorna el valor asociado con la clave k del mapeo, o null si no hay una entrada con clave k }

```
t ← A[h(k)].remove(k) { Delega en el mapeo implementado con lista. }
if t ≠ null then { k fue encontrado }
    n  n – 1 { decremento la cantidad de entradas del mapeo }
return t
```

**Algoritmo** get(k)
{ retorna el valor asociado con la clave k en el mapeo, o null si no hay una entrada con clave k en el mapeo }
    **return** A[h(k)].get(k) { Delega en el get del mapeo implementado con lista. }

**Algoritmo** put(k, v)
{ Si hay una entrada en el mapeo con clave k, reemplaza el valor con v y retorna el viejo valor. Sino agrega la entrada (k,v) y retorna null }
    t ← A[h(k)].put(k,v) { Delega en el put del mapeo implementado con lista. }
    if t = null then { k es una nueva clave }
        n ← n+1 { incremento la cantidad de entradas en el mapeo }
    **return** t

**Algoritmo** remove(k)
{ Retorna el valor asociado con la clave k del mapeo, o null si no hay una entrada con clave k }
    t ← A[h(k)].remove(k) { Delega en el mapeo implementado con lista. }
    if t ≠ null then { k fue encontrado }
        n  n – 1 { decremento la cantidad de entradas del mapeo }
    **return** t

# Factor de Carga y Rehashing

- Se llama **factor de carga** a la relación entre el número total de elementos (*n)* y la cantidad de cubetas (*C*), o sea *n/C*
- Es deseable que este factor de carga se mantenga cercano o por debajo de 1

# Factor de Carga y Rehashing

- Se llama **factor de carga** a la relación entre el número total de elementos (*n)* y la cantidad de cubetas (*C*), o sea *n/C*
- Es deseable que este factor de carga se mantenga cercano o por debajo de 1
- En un hash abierto, un mayor factor de carga significa listas más largas en cada cubeta → hay que buscar entre más cosas.

# Factor de Carga y Rehashing

- Si el factor de carga de una tabla hash supera significativamente un umbral especificado,
  - se puede redimensionar la tabla (para recuperar el factor de carga especificado),
  - todos los objetos se deben insertar en esta nueva tabla.
  - A esto se lo llama **rehashing.**

# Factor de Carga y Rehashing

- Al hacer un *rehashing*, se puede aumentar el tamaño del nuevo arreglo (cubetas)
  - cerca del doble del tamaño anterior
  - igual a un número primo
- Una vez asignado este nuevo arreglo de contenedores, debemos definir una nueva función hash que lo acompañe.

# Factor de Carga y Rehashing

Por ejemplo, Java provee la clase HashMap con un umbral de carga de 0.75 (https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html)

# Tablas de Hash

Tabla de Hash (o Tabla Hash) es una estrategia para lograr esto.

Tenemos dos variantes de la estrategia, Hash Abierto y Hash Cerrado.

# Tablas de Hash

Tabla de Hash (o Tabla Hash) es una estrategia para lograr esto.

Tenemos dos variantes de la estrategia, Hash Abierto y **Hash Cerrado**.

# Tablas de Hash Cerrado

La tabla de hash cerrado simplemente cuenta con un arreglo;

En cada bucket se encuentra directamente una entrada o un nulo;

Cuando se produce una colisión, se busca el siguiente lugar disponible hacia abajo de manera circular;

# Tablas de Hash Cerrado

La tabla de hash cerrado simplemente cuenta con un arreglo;

En cada bucket se encuentra directamente una entrada o un nulo;

Cuando se produce una colisión, se busca el siguiente lugar disponible hacia abajo de manera circular;

¿Por qué considerar esto?

# Tablas de Hash Cerrado

La tabla de hash cerrado simplemente cuenta con un arreglo;

En cada bucket se encuentra directamente una entrada o un nulo;

Cuando se produce una colisión, se busca el siguiente lugar disponible hacia abajo de manera circular;

¿Por qué considerar esto?

- Eficiencia de caché
- Eficiencia de memoria
- Manejo de espacio

# Tabla de Hash Cerrado para Mapeo/Diccionario

Tracemos nuevamente este ejemplo pero ahora con hash cerrado:

M  = new MapeoConHashCerrado<Integer,Character>(); //Supongamos que se implementa con 10 Cubetas

M.put( 5, ‘A’ )

M.put( 6, ‘B’ )

M.put( 3, ‘C’ )

M.put( 15, ‘D’ )

M.put( 25, ‘E’ )

M.put( 35, ‘F’ )

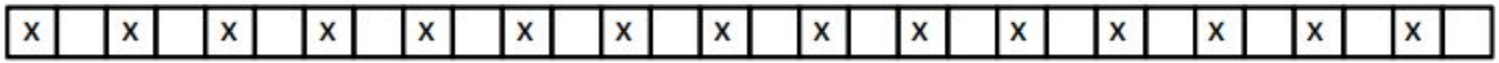
M.put( 56, ‘G’ )

M.put( 23, ‘H’ )

M.get(15) ; M.get(40); M.remove(15)

# Tabla de Hash cerrado: factor de carga

- Para evitar que las búsquedas se hagan muy lentas, en el hash cerrado se buscar factores de carga más bajos
- En nuestro libro se sugiere $\lambda = n/N < 0.5$
- En teoría buscaríamos algo de este estilo



- get, put y remove tardarían máximo dos pasos en ejecutarse

# Tabla de Hash cerrado: políticas de resolución de colisiones

- **Resolución lineal**: Si i=h(k), prueba iterativamente los buckets A[(i+f(j)) mod N] para j=0,1,2,... donde f(j)=j hasta que encuentra un bucket vacío. Ej: si h(k)=4, prueba en los buckets 4,5,6,7,...
- **Resolución cuadrática**: Si i=h(k), prueba iterativamente los buckets A[(i+f(j)) mod N] para j=0,1,2,... donde $f(j)=j^2$ hasta que encuentra un bucket vacío. Si h(k)=4, prueba en los buckets $4+0$, $4+1^2$, $4+2^2$, $4+3^2$, ...
- **Ventaja**: Evita los patrones de agrupamiento de hash lineal pero crea otro tipo que se llama “agrupamiento secundario”
- **Problema**: Si N no es primo, puede no encontrar un slot vacío aunque exista
- **Problema**: Si N es primo y el arreglo está lleno a la mitad o más (i>N/2), no va a encontrar un slot vacío por más que éste exista

# Tabla de Hash cerrado: políticas de resolución de colisiones

Hashing doble

- Si i=h(k), y A[i] está ocupado, se usa una segunda función de hash h’ tal que prueba los buckets A[(i + f(j)) mod N] para j=1,2,3... donde f(j) = j*h’(k).
  - La función h’(k) no puede evaluar a 0
  - Se usa h’(k) = q – (k mod q) para un primo q < N
  - N debería ser primo

# Comparación hash abierto y cerrado

Abierto

- Simpleza y flexibilidad: La tabla nunca se llena, ni necesita reorganizar
- Las colisiones solo lo vuelven más lento
- En pruebas experimentales el hash abierto es más eficiente

Cerrado

- Puede ahorrar algo de espacio por el overhead de las listas
- Puede permitir algo de eficiencia en el acceso con el cache en memoria

# TDA Mapeo, Diccionario y Conjunto

Capítulo 9 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.

# THE END.